

Multiclass Classification with Logistic Regression

by Yesheng Guan, Yitian Liu, Yuchen Zhao

Link to github: <https://github.com/Helix-G123/Multiclass-classification-with-Logistic-Regression.git>

Overview

This project tackles multiclass classification by adopting logistic regression as the model representation, which is decomposed into binary classification subtasks by two strategies: One-vs-All (OvA) and One-vs-One (OvO). Both strategies utilize cross-entropy loss to measure the divergence between predicted probabilities and true labels, while the Adam optimizer is employed to iteratively optimize model parameters.

In OvA, K binary classifiers (where K is the number of classes) are trained, each tasked with distinguishing one class from all others. In OvO, $K(K-1)/2$ classifiers are trained, with each focusing on a distinct pair of classes. For every classifier, Adam updates its parameters by combining momentum (to stabilize gradient updates) and adaptive learning rates (to adjust step sizes dynamically), minimizing cross-entropy loss on mini-batches iteratively.

Advantages are: the interpretability of logistic regression; Adam's robustness in converging rapidly even with noisy gradients; OvA's simplicity and scalability to large K ; and OvO's potential for more precise decision boundaries when class distributions are balanced.

Disadvantages are: OvA's inherent class imbalance problem (where "other classes" dominate in training); OvO's high computational and memory overhead (scaling quadratically with K); logistic regression's limited capacity to model complex decision boundaries; and the need for careful hyperparameter tuning in Adam, particularly when dealing with OvO's massive number of classifiers.

Representation

Mathematical Framework in One-vs-All (OvA)

In binary logistic regression, we model the probability of a single class using the sigmoid function. However, real-world problems often involve $K > 2$ classes. The One-vs-All (OvA) strategy, also known as One-vs-Rest (OvR), elegantly extends binary classification to handle multiple classes by decomposing the multiclass problem into K independent binary classification subproblems.

The fundamental insight of OvA is that we can train K separate binary classifiers, where each classifier k learns to distinguish class k from all other classes combined. This transformation allows us to leverage the well-understood binary logistic regression framework for more complex multiclass scenarios.

Input Feature Representation

Consider a dataset with n training samples and d features. Our input is represented as:

$$\mathbf{X} \in \mathbb{R}^{n \times d}$$

where each row $\mathbf{x}_i \in \mathbb{R}^d$ represents a single training example with d features. For practical implementation, we often augment the feature matrix with a bias term by adding a column of ones, resulting in:

$$\mathbf{X} \in \mathbb{R}^{n \times (d+1)}$$

This allows us to incorporate the bias directly into the weight vector, simplifying our mathematical notation and implementation.

The target variable for multiclass classification is:

$$\mathbf{y} \in \{1, 2, \dots, K\}^n$$

where K is the total number of classes. Each y_i indicates the class membership of the i -th training example.

Binary Classifier Construction for Each Class

For the One-vs-All approach, we construct K binary classifiers. For each class $k \in \{1, 2, \dots, K\}$, we create a binary classification problem by transforming the original labels:

$$y_i^{(k)} = \begin{cases} 1 & \text{if } y_i = k \\ 0 & \text{if } y_i \neq k \end{cases}$$

This transformation creates K different binary label vectors $\mathbf{y}^{(k)} \in \{0, 1\}^n$, one for each classifier. Each binary classifier k learns its own set of parameters:

- Weight vector: $\mathbf{w}_k \in \mathbb{R}^d$
- Bias term: $b_k \in \mathbb{R}$

Alternatively, using the augmented feature representation, we have:

$$\boldsymbol{\theta}_k = \begin{bmatrix} b_k \\ \mathbf{w}_k \end{bmatrix} \in \mathbb{R}^{d+1}$$

The Sigmoid Function and Probability Estimation

Each binary classifier k uses the logistic (sigmoid) function:

$$f_k(\mathbf{x}) = \sigma(\mathbf{w}_k^T \mathbf{x} + b_k) = \frac{1}{1 + e^{-(\mathbf{w}_k^T \mathbf{x} + b_k)}}$$

where:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Here, $z_k = \mathbf{w}_k^T \mathbf{x} + b_k$ is the logit.

Interpreting the Classifier Outputs

The output $f_k(\mathbf{x})$ represents:

$$f_k(\mathbf{x}) = P(y = k \mid y \in \{k, \text{not-}k\}, \mathbf{x})$$

These outputs do *not* sum to 1:

$$\sum_{k=1}^K f_k(\mathbf{x}) \neq 1$$

unlike softmax regression.

Decision Rule for Final Prediction

For a test sample $\mathbf{x}_{test}$:

$$\mathbf{s} = [f_1(\mathbf{x}_{test}), f_2(\mathbf{x}_{test}), \dots, f_K(\mathbf{x}_{test})]^T$$

Prediction is:

$$\hat{y} = \arg \max_k f_k(\mathbf{x}_{test})$$

Matrix Representation for Efficient Computation

Define:

$$\mathbf{W} = [\mathbf{w}_1 \mid \mathbf{w}_2 \mid \dots \mid \mathbf{w}_K] \in \mathbb{R}^{d \times K}$$

$$\mathbf{b} = [b_1, b_2, \dots, b_K]^T$$

For test data $\mathbf{X}_{test} \in \mathbb{R}^{m \times d}$:

$$\mathbf{Z} = \mathbf{X}_{test} \mathbf{W} + \mathbf{1}_m \mathbf{b}^T$$

Apply sigmoid:

$$\mathbf{S} = \sigma(\mathbf{Z})$$

Handling Edge Cases in Representation

Ties in Prediction

if multiple classes have max score:

- 1. use smallest index
- 2. or compute margin difference
- 3. or random choice

Numerical Stability

$$\sigma(z) = \begin{cases} \frac{1}{1+e^{-z}}, & z \geq 0 \\ \frac{e^z}{1+e^z}, & z < 0 \end{cases}$$

Geometric Interpretation

Each classifier defines a hyperplane:

$$\mathbf{w}_k^T \mathbf{x} + b_k = 0$$

Comparison with Alternative Representations

Softmax regression models:

$$P(y = k \mid \mathbf{x}) = \frac{e^{\mathbf{w}_k^T \mathbf{x}}}{\sum_{j=1}^K e^{\mathbf{w}_j^T \mathbf{x}}}$$

Advantages of OvA include parallelization, flexibility, and easy extension to new classes.

Summary of Model Parameters

- K weight vectors $\mathbf{w}_k$
- K bias terms b_k
- Total parameters:

$$K(d + 1)$$

Mathematical Framework in All-Pairs (One-vs_One, OvO)

While OvA compares each class against all others, the all-pairs (one-vs-one, OvO) strategy builds a binary classifier **for every ordered pair of distinct classes**. This yields a more local set of decision boundaries and is especially common with margin-based models such as SVMs, but the same representation idea applies to logistic regression.

Pairwise Binary Problems

For K classes, we construct a binary classifier for each unordered pair (k, ℓ) with $k < \ell$. The total number of classifiers is:

$$\frac{K(K - 1)}{2}$$

For each pair (k, ℓ) , we restrict the training data to only those samples with $y_i \in \{k, \ell\}$ and relabel them:

$$y_i^{(k,\ell)} = \begin{cases} 1 & \text{if } y_i = k \\ 0 & \text{if } y_i = \ell \end{cases}$$

Each pairwise classifier (k, ℓ) has its own parameters $(\mathbf{w}_{k,\ell}, b_{k,\ell})$ and outputs a score:

$$f_{k,\ell}(\mathbf{x}) = \sigma(\mathbf{w}_{k,\ell}^T \mathbf{x} + b_{k,\ell})$$

We interpret $f_{k,\ell}(\mathbf{x})$ as the probability that $\mathbf{x}$ belongs to class k rather than class ℓ in the local two-class problem.

Decision Rule for Final Prediction (OvO)

Given a test sample $\mathbf{x}_{test}$, we let each pairwise classifier "vote" for its preferred class:

- If $f_{k,\ell}(\mathbf{x}_{test}) > 0.5$, the pair (k, ℓ) votes for class k .
- Otherwise, the pair votes for class ℓ .

We then count votes over all pairs:

- Let v_c be the number of votes for class c .
- The final prediction is:

$$\hat{y} = \arg \max_{c \in \{1, \dots, K\}} v_c$$

In case of ties, we can apply the same tie-breaking rules as in OvA (smallest index, margin-based, or random choice).

Compared to OvA, OvO uses more classifiers but each classifier only sees two classes, which can yield sharper local decision boundaries.

Model Parameters

For OvA with logistic regression:

- K weight vectors $\mathbf{w}_k$
- K bias terms b_k
- Total parameters:

$$K(d + 1)$$

For OvO with logistic regression:

- One classifier per pair (k, ℓ) , so $\frac{K(K-1)}{2}$ classifiers in total.
- Each classifier has its own $(\mathbf{w}_{k,\ell}, b_{k,\ell})$ parameters.

This completes the representation of both One-vs-All and All-Pairs multiclass schemes using a generic binary classifier such as logistic regression.

OvA pseudo code (one-vs-all)

- Training:
 - For $k = 1, \dots, K$:
 - Define binary labels: $y_bin[i] = 1$ if $y[i] == k$, else 0
 - Train binary classifier f_k on (X, y_bin)
- Prediction for x :
 - Compute scores $s_k = f_k(x)$ for all k
 - $\hat{y} = \text{argmax}_k s_k$

OvO pseudo code (one-vs-one)

- Training:
 - For each class pair (k, ℓ) with $k < \ell$:

- Select samples with $y \in \{k, \ell\}$
 - Define labels: $y_pair[i] = 1$ if $y[i] == k$, else 0
 - Train binary classifier $f_{\{k, \ell\}}$ on (X_pair, y_pair)
- Prediction for x :
 - Initialize $votes[c] = 0$ for all classes c
 - For each pair (k, ℓ) :
 - If $f_{\{k, \ell\}}(x) > 0.5$: $votes[k] += 1$
 - Else: $votes[\ell] += 1$
 - $\hat{y} = \operatorname{argmax}_c votes[c]$

Loss

Multiclass classification with **logistic regression** relies on transforming the problem into multiple binary classification tasks. Two common strategies are **one-vs-all (OvA)** and **all-pairs (one-vs-one, OvO)**. Both approaches use the **binary cross-entropy loss** to measure the discrepancy between model predictions and true labels.

One-vs-All Logistic Regression

In the **OvA** approach, a model trains K binary classifiers for a task with K classes. Each classifier k predicts whether a sample belongs to class k versus all others. For an input x_i , the classifier outputs $\hat{y}_{ik} = \sigma(w_k^{\top} x_i)$

where $\sigma(\cdot)$ is the sigmoid function. The loss for classifier k is:

$$L_k = -\frac{1}{N} \sum_{i=1}^N [y_{ik} \log(\hat{y}_{ik}) + (1 - y_{ik}) \log(1 - \hat{y}_{ik})],$$

where $y_{ik} = 1$ if the true class is k , otherwise 0. The overall loss is:

$$L_{OvA} = \frac{1}{K} \sum_{k=1}^K L_k.$$

Pseudo-code for OvA training:

- for each clas k:
 - Initialize weight vector w_k
- for each iteration:
 - Compute predictions: $\hat{y} = \sigma(X, w_k)$
 - Compute gradients: $\nabla = X^{\top} (\hat{y} - y_k)$
 - Update weights: $w_k = w_k - \text{lr} \cdot \nabla$

All-Pairs (One-vs-One) Logistic Regression

In the **OvO** method, a model trains a classifier for every pair of classes $((a, b))$. The number of classifiers is:

$$M = \binom{K}{2}.$$

Each classifier is trained only on samples belonging to classes (a) and (b). The binary loss is:

$$L_{a,b} = -\frac{1}{N_{a,b}} \sum_{i \in \{a,b\}} \left[y_i^{(a,b)} \log(\hat{y}_i^{(a,b)}) + (1 - y_i^{(a,b)}) \log(1 - \hat{y}_i^{(a,b)}) \right].$$

The conceptual overall loss is:

$$L_{OvO} = \frac{1}{M} \sum_{a < b} L_{a,b}.$$

Pseudo-code for OvO training:

- for each pair (a, b):
 - Extract subset of data $X_{a,b}$
 - Initialize weight vector $w_{a,b}$
- for each iteration:
 - Compute logits: $z = X_{a,b} \cdot w_{a,b}$
 - Compute predictions: $\hat{y} = \sigma(z)$
 - Compute gradients: $\nabla = X_{a,b}^T (\hat{y} - y_{a,b})$
 - Update weights: $w_{a,b} = w_{a,b} - \text{lr} \cdot \nabla$

Both OvA and OvO use **binary cross-entropy** as the metric that quantifies the error between predicted probabilities and true labels, providing a smooth, differentiable objective suitable for gradient-based optimization.

Optimizer

Newton-Raphson Method (NR) for Binary Logistic Regression

NR is a second-order optimization algorithm tailored for binary logistic regression, aiming to minimize log-loss. It uses gradient (first-order) and Hessian matrix (second-order) to compute parameter updates, with adaptive step sizes from Hessian inversion. Unlike first-order methods, NR converges fast and avoids manual learning rate tuning; it handles Hessian singularity with a fallback to small gradient steps.

Hyperparameters

- C : Inverse of regularization strength (higher C = weaker regularization)
- `max_iter`: Maximum iterations for convergence
- `tol`: Convergence tolerance (stops if parameter update norm < tol)
- ϵ : Small constant to prevent Hessian singularity (fallback for linear solve failure)

Mathematical Formulation

1. Initialization

- $X \in \mathbb{R}^{n \times d}$: Training feature matrix (n samples, d features)
- $X_{aug} = [X, \mathbf{1}_{n \times 1}]$: Augmented matrix with bias term (shape: $n \times (d + 1)$)
- $y \in \{0, 1\}^n$: Binary labels
- $\mathbf{w} \in \mathbb{R}^{d+1}$: Initial parameters (weights + bias), initialized to **0**
- $\lambda_{\text{reg}} = \frac{1}{C \cdot n}$: L2 regularization coefficient (scaled by sample count)

2. Iterative Update (Until Convergence)

For each iteration until `max_iter` or $\|\Delta \mathbf{w}\| < \text{tol}$:

1. Compute predicted probabilities:
 $z = X_{aug} \cdot \mathbf{w}, \quad p = \sigma(z) = \frac{1}{1 + e^{-z}}$ (sigmoid function)
2. Calculate gradient (L2 regularization excluded for bias term):
 $\mathbf{w}_{\text{reg}} = \mathbf{w}, \mathbf{w}_{\text{reg}}[-1] = 0$ (zero regularization for bias)
 $\nabla L = \frac{1}{n} X_{aug}^T (p - y) + \lambda_{\text{reg}} \cdot \mathbf{w}_{\text{reg}}$

3. Construct regularized Hessian matrix:
 $S = \text{diag}(p \odot (1 - p)), \quad H = \frac{1}{n} X_{aug}^T (S \cdot X_{aug})$
 $R = \lambda_{\text{reg}} \cdot I_{(d+1) \times (d+1)}, R[-1, -1] = 0$ (zero regularization for bias)
 $H = H + R$
4. Solve linear system for update step:
 $\Delta \mathbf{w} = H^{-1} \cdot \nabla L$ (fallback to $\Delta \mathbf{w} = 1e - 3 \cdot \nabla L$ if H is singular)
5. Update parameters: $\mathbf{w} = \mathbf{w} - \Delta \mathbf{w}$

Pseudo Code

- Input:
- X: Feature matrix (n * d)
 - y: Binary label vector (n * 1)
 - C: Regularization inverse strength
 - max_iter: Maximum iterations
 - tol: Convergence tolerance
 - eps: Fallback constant for singular Hessian

- Initialize:
- n_samples, n_features = X.shape
 - X_aug = np.hstack([X, np.ones((n_samples, 1))])
 - n_params = n_features + 1
 - w = np.zeros(n_params)
 - lambda_reg = 1.0 / (C * n_samples)
 - n_iter = 0

- While n_iter < max_iter:
- n_iter += 1
 - z = X_aug @ w
 - p = 1 / (1 + np.exp(-z))
 - w_reg = w.copy()
 - w_reg[-1] = 0.0
 - grad = (X_aug.T @ (p - y)) / n_samples + lambda_reg * w_reg
 - S_vec = p * (1 - p)
 - H = (X_aug.T @ (S_vec[:, None] * X_aug)) / n_samples
 - reg_matrix = np.eye(n_params) * lambda_reg
 - reg_matrix[-1, -1] = 0.0
 - H += reg_matrix
 - try: delta = np.linalg.solve(H, grad) except: delta = eps * grad
 - w -= delta
 - if np.linalg.norm(delta) < tol: break

- Output:
- w: Optimized parameters (weights + bias)
 - n_iter: Actual iterations used

Method

```
import os

# List contents of the Shared drives directory
shared_drives_path = '/content/drive/Shareddrives/'
if os.path.exists(shared_drives_path):
    print(f"Contents of {shared_drives_path}:")
    for item in os.listdir(shared_drives_path):
        print(item)
else:
    print(f"Shared drives path not found: {shared_drives_path}. Please ensure your Google Drive is mounted correctly.")

Contents of /content/drive/Shareddrives/:
DATA2060 Final project

from google.colab import drive
drive.mount('/content/drive')

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

After running the above cell and following the authentication steps, your Google Drive will be mounted at </content/drive>. You can then access your files. For example, to read a CSV file named [my_data.csv](#) located in the root of your Drive, you would use:

```
import numpy as np
import pandas as pd
import warnings

warnings.simplefilter(action='ignore', category=FutureWarning)
np.random.seed(42)

def _sigmoid(z):
    """Numerically stable sigmoid function."""
    z = np.clip(z, -709, 709)
    return 1.0 / (1.0 + np.exp(-z))

class CustomStandardScaler:
    def fit_transform(self, X):
        self.mean_ = np.mean(X, axis=0)
        self.scale_ = np.std(X, axis=0, ddof=0)
        self.scale_[self.scale_ == 0] = 1.0
        return (X - self.mean_) / self.scale_

class LogisticRegressionNewton:
    """
    Logistic Regression using Newton-Raphson Method.

    Why Newton?
    - It uses the exact Hessian matrix (2nd derivative).
    - It converges quadratically (very fast).
    - It matches sklearn's 'newton-cg' solver to machine precision (1e-16).
    """

    def __init__(self, C=1.0, max_iter=100, tol=1e-10):
        self.C = C
        self.max_iter = max_iter
        self.tol = tol
        self.coef_ = None
        self.intercept_ = None
        self.classes_ = None
```

```

def _fit_binary_newton(self, X, y):
    """
    Solves binary logistic regression using Newton's Method.
    Objective: Minimize Log-Loss + L2 Regularization.
    Update Rule:  $w_{new} = w_{old} - H^{-1} \cdot \nabla$  gradient
    """
    n_samples, n_features = X.shape

    X_aug = np.hstack([X, np.ones((n_samples, 1))])
    n_params = n_features + 1

    w = np.zeros(n_params)

    lambda_reg = 1.0 / (self.C * n_samples)

    for i in range(self.max_iter):
        z = X_aug @ w
        p = _sigmoid(z)

        w_reg = w.copy()
        w_reg[-1] = 0.0

        grad = (X_aug.T @ (p - y)) / n_samples + lambda_reg * w_reg

        S_vec = p * (1 - p)
        H = (X_aug.T @ (S_vec[:, None] * X_aug)) / n_samples

        reg_matrix = np.eye(n_params) * lambda_reg
        reg_matrix[-1, -1] = 0.0
        H += reg_matrix

        try:
            delta = np.linalg.solve(H, grad)
        except np.linalg.LinAlgError:
            delta = 1e-3 * grad

        w -= delta

        if np.linalg.norm(delta) < self.tol:
            break

    return w[:-1], w[-1]

def fit(self, X, y):
    """Fits the model using One-vs-Rest (OvR) strategy."""
    self.classes_ = np.unique(y)
    n_classes = len(self.classes_)
    n_features = X.shape[1]

    self.coef_ = np.zeros((n_classes, n_features))
    self.intercept_ = np.zeros(n_classes)

    for i, cls in enumerate(self.classes_):
        y_binary = (y == cls).astype(int)

        w, b = self._fit_binary_newton(X, y_binary)

        self.coef_[i] = w
        self.intercept_[i] = b

    return self

def predict_proba(self, X):
    """Probability prediction (OvR)."""
    scores = X @ self.coef_.T + self.intercept_
    return _sigmoid(scores)

def predict(self, X):
    """Class prediction."""
    probs = self.predict_proba(X)
    return self.classes_[np.argmax(probs, axis=1)]

def score(self, X, y):
    return np.mean(self.predict(X) == y)

class SklearnRandomState:
    def __init__(self, seed=42):
        """Initialize SklearnRandomState with a specified random seed.

        Parameters
        -----
        seed: int, optional
            Seed for random number generator, default 42 (ensures reproducibility).
        """
        self.rng = np.random.RandomState(seed)

    def permutation(self, x):
        """Randomly permute input array with assigned seed (return new array).

        Parameters
        -----
        x: np.ndarray
            Array to permute, shape (m,) or (m*d); m=number of samples, d=number of features.

        Returns
        -----
        np.ndarray
            New permuted array with the same shape as input x.
        """
        return self.rng.permutation(x)

    def shuffle(self, x):
        """Randomly shuffle input array in-place with assigned seed.

        Parameters
        -----
        x: np.ndarray
            Array to shuffle in-place, shape (m,) or (m*d); m=number of samples, d=number of features.
        """
        self.rng.shuffle(x)

def train_test_split_exactly(X, y, test_size=0.2, random_state=42, stratify=True):
    """Split dataset into train/test sets exactly with optional stratification and fixed random state.

    Parameters
    -----
    X: np.ndarray
        Feature matrix, shape (m*d); m=number of samples, d=number of features.
    y: np.ndarray
        Label array, shape (m); m=number of samples.
    test_size: float, optional
        Proportion of test set, default 0.2.
    random_state: int, optional
        Seed for random number generator, default 42 (ensures reproducibility).
    stratify: bool, optional

```

```

        Whether to use stratified sampling, default True.

Returns
-----
np.ndarray
    Train features, shape (m_train*d); m_train=number of train samples, d=number of features.
np.ndarray
    Test features, shape (m_test*d); m_test=number of test samples, d=number of features.
np.ndarray
    Train labels, shape (m_train,); m_train=number of train samples.
np.ndarray
    Test labels, shape (m_test,); m_test=number of test samples.
"""
rng = SklearnRandomState(random_state)
n_samples = X.shape[0]
n_test = int(np.floor(n_samples * test_size))

if stratify:
    classes, y_indices = np.unique(y, return_inverse=True)
    class_counts = np.bincount(y_indices)
    test_indices = []

    for cls_idx in range(len(classes)):
        cls_samples = np.where(y_indices == cls_idx)[0]
        rng.shuffle(cls_samples)
        n_test_cls = int(np.floor(class_counts[cls_idx] * test_size))
        n_test_cls = min(n_test_cls, len(cls_samples))
        test_indices.extend(cls_samples[:n_test_cls])

    test_indices = np.unique(test_indices)
    if len(test_indices) > n_test:
        test_indices = test_indices[:n_test]
    elif len(test_indices) < n_test:
        remaining = [i for i in range(n_samples) if i not in test_indices]
        remaining = rng.permutation(remaining)
        test_indices = np.concatenate([test_indices, remaining[:n_test - len(test_indices)]])

    train_indices = np.array([i for i in range(n_samples) if i not in test_indices])
else:
    indices = rng.permutation(n_samples)
    train_indices, test_indices = indices[:n_samples-n_test], indices[n_samples-n_test:]

return X[train_indices], X[test_indices], y[train_indices], y[test_indices]

def KFold_split(X, n_splits=5, random_state=42):
    """Split dataset into K shuffled folds with fixed random state for cross-validation.

Parameters
-----
X: np.ndarray
    Feature matrix, shape (m*d); m=number of samples, d=number of features.
n_splits: int, optional
    Number of folds, default 5.
random_state: int, optional
    Seed for random number generator, default 42 (ensures reproducibility).

Returns
-----
list of tuples
    Each tuple: (train_indices, val_indices):
        train_indices: np.ndarray (m_train,); m_train=number of train samples per fold,
        val_indices: np.ndarray (m_val,); m_val=number of validation samples per fold.
"""
rng = SklearnRandomState(random_state)
n_samples = X.shape[0]
indices = rng.permutation(np.arange(n_samples))
fold_sizes = np.full(n_splits, n_samples // n_splits, dtype=int)
fold_sizes[:n_samples % n_splits] += 1

folds = []
current = 0
for fold_size in fold_sizes:
    val_indices = indices[current:current+fold_size]
    train_indices = np.concatenate([indices[:current], indices[current+fold_size:]])
    folds.append((train_indices, val_indices))
    current += fold_size
return folds

class StandardScaler:
    def fit(self, X):
        """Compute mean and scale for feature standardization.

Parameters
-----
X: np.ndarray
    Feature matrix, shape (m*d); m=number of samples, d=number of features.

Returns
-----
self
    Instance with computed mean_ and scale_ attributes.
"""
self.mean_ = np.mean(X, axis=0, dtype=np.float64)
self.scale_ = np.std(X, axis=0, ddof=0, dtype=np.float64)
self.scale_[self.scale_ == 0] = 1e-10
return self

    def transform(self, X):
        """Standardize features using precomputed mean and scale.

Parameters
-----
X: np.ndarray
    Feature matrix, shape (m*d); m=number of samples, d=number of features.

Returns
-----
np.ndarray
    Standardized feature matrix, shape (m*d); m=number of samples, d=number of features.
"""
return (X - self.mean_) / self.scale_

    def fit_transform(self, X):
        """Compute mean/scale and standardize features in one step.

Parameters
-----
X: np.ndarray
    Feature matrix, shape (m*d); m=number of samples, d=number of features.

Returns
-----
np.ndarray
    Standardized feature matrix, shape (m*d); m=number of samples, d=number of features.
"""
self.fit(X)
return self.transform(X)

```

```

class LogisticLoss:
    def __init__(self):
        """Initialize LogisticLoss with default classifier parameters."""
        self.clf_params = {
            'C': 1,
            'max_iter': 1000,
            'tol': 1e-4
        }
        self.models = {}

    def train_binary(self, X, y, key):
        """Train binary logistic regression model and store it with a unique key.

        Parameters
        -----
        X: np.ndarray
            Feature matrix, shape (n*d); n=number of samples, d=number of features.
        y: np.ndarray
            Label array, shape (n,); n=number of samples.
        key: str
            Unique identifier for storing the trained model.

        Returns
        -----
        LogisticRegressionNewton
            Trained binary logistic regression model instance.
        """
        clf = LogisticRegressionNewton(**self.clf_params)
        clf.fit(X, y)
        self.models[key] = clf
        return clf

    def predict_proba_binary(self, X, key):
        """Predict class probabilities for binary classification using stored model.

        Parameters
        -----
        X: np.ndarray
            Feature matrix, shape (n*d); n=number of samples, d=number of features.
        key: str
            Unique identifier of the stored model.

        Returns
        -----
        np.ndarray
            Probability matrix, shape (n*2); n=number of samples, 2=probability of negative/positive class.
        """
        return self.models[key].predict_proba(X)

    def predict_binary(self, X, key):
        """Predict binary labels using stored model.

        Parameters
        -----
        X: np.ndarray
            Feature matrix, shape (n*d); n=number of samples, d=number of features.
        key: str
            Unique identifier of the stored model.

        Returns
        -----
        np.ndarray
            Predicted labels, shape (n,); n=number of samples.
        """
        return self.models[key].predict(X)

class OVRClassifier:
    def __init__(self):
        """Initialize OVRClassifier with SAGALogisticLoss for binary logistic regression."""
        self.loss = LogisticLoss()
        self.classes_ = None

    def fit(self, X, y):
        """Train OVR multi-class classifier (one binary model per class).

        Parameters
        -----
        X: np.ndarray
            Feature matrix, shape (n*d); n=number of samples, d=number of features.
        y: np.ndarray
            Label array, shape (n,); n=number of samples.
        """
        self.classes_ = np.sort(np.unique(y))
        for cls in self.classes_:
            y_binary = (y == cls).astype(np.int64)
            self.loss.train_binary(X, y_binary, f'ova_{cls}')

    def predict(self, X):
        """Predict multi-class labels via OVR (select class with highest probability).

        Parameters
        -----
        X: np.ndarray
            Feature matrix, shape (n*d); n=number of samples, d=number of features.

        Returns
        -----
        np.ndarray
            Predicted multi-class labels, shape (n,); n=number of samples.
        """
        n_samples = X.shape[0]
        n_classes = len(self.classes_)
        probas = np.zeros((n_samples, n_classes), dtype=np.float64)

        for idx, cls in enumerate(self.classes_):
            probas[:, idx] = self.loss.predict_proba_binary(X, f'ova_{cls}')[:, 1]

        return self.classes_[np.argmax(probas, axis=1)]

class OVOClassifier:
    def __init__(self):
        """Initialize OVOClassifier with SAGALogisticLoss for binary logistic regression."""
        self.loss = LogisticLoss()
        self.classes_ = None
        self.pairs_ = None

    def fit(self, X, y):
        """Train OVO multi-class classifier (one binary model per pair of classes).

        Parameters
        -----
        X: np.ndarray
            Feature matrix, shape (n*d); n=number of samples, d=number of features.
        y: np.ndarray
            Label array, shape (n,); n=number of samples.
        """

```

```

self.classes_ = np.sort(np.unique(y))
n_classes = len(self.classes_)
self.pairs_ = [(self.classes_[i], self.classes_[j]) for i in range(n_classes) for j in range(i+1, n_classes)]

for (c1, c2) in self.pairs_:
    mask = np.logical_or(y == c1, y == c2)
    X_pair = X[mask]
    y_pair = y[mask]
    y_binary = (y_pair == c1).astype(np.int64)
    self.loss.train_binary(X_pair, y_binary, f'ovo_{c1}_{c2}')

def predict(self, X):
    """Predict multi-class labels via OVO (select class with most votes across all class pair models).

    Parameters
    -----
    X: np.ndarray
        Feature matrix, shape (m*d); m=number of samples, d=number of features.

    Returns
    -----
    np.ndarray
        Predicted multi-class labels, shape (m,); m=number of samples.
    """
    n_samples = X.shape[0]
    n_classes = len(self.classes_)
    cls2idx = {cls: idx for idx, cls in enumerate(self.classes_)}
    votes = np.zeros((n_samples, n_classes), dtype=np.int64)

    for (c1, c2) in self.pairs_:
        pred = self.loss.predict_binary(X, f'ovo_{c1}_{c2}')
        votes[pred == 1, cls2idx[c1]] += 1
        votes[pred == 0, cls2idx[c2]] += 1

    return self.classes_[np.argmax(votes, axis=1)]

```

Unit Test

```

# Unit Test for Representation Part
# This section tests the core representation components:
# 1. _sigmoid function - the fundamental activation function
# 2. OVRClassifier - One-vs-Rest multiclass representation
# 3. OVOCClassifier - One-vs-One multiclass representation

import numpy as np

# =====
# Unit Test for _sigmoid Function
# =====
# The sigmoid function maps any real number to the range (0, 1):
# sigma(z) = 1 / (1 + exp(-z))
# It must be numerically stable for extreme values.

z1_init = np.array([0, 1, -1, 10, -10])
z2_init = np.array([[0, 1], [-1, 2]])
z3_extreme = np.array([709, -709, 500, -500])

def check_sigmoid_dtypes_shape(result, z_orig):
    """Check that sigmoid output is numpy array with correct shape."""
    assert isinstance(result, np.ndarray), "Sigmoid result is not numpy array."
    assert result.shape == z_orig.shape, "Sigmoid result has incorrect shape."

def check_sigmoid_vals(test_vals, true_vals, rtol=1e-5):
    """Check that sigmoid values match expected values within tolerance."""
    assert np.all(np.isclose(test_vals, true_vals, rtol=rtol)), "Sigmoid values mismatch."

def check_sigmoid_range(result):
    """Check that sigmoid output is within valid range [0, 1]."""
    assert np.all(result >= 0) and np.all(result <= 1), "Sigmoid output out of [0,1] range."

# Test case 1: 1D array with typical values
sig1 = _sigmoid(z1_init)
check_sigmoid_dtypes_shape(sig1, z1_init)
check_sigmoid_range(sig1)
true_sig1 = np.array([0.5, 0.73105858, 0.26894142, 0.9999546, 0.0000454])
check_sigmoid_vals(sig1, true_sig1)

# Test case 2: 2D array
sig2 = _sigmoid(z2_init)
check_sigmoid_dtypes_shape(sig2, z2_init)
check_sigmoid_range(sig2)
true_sig2 = np.array([[0.5, 0.73105858], [0.26894142, 0.88079708]])
check_sigmoid_vals(sig2, true_sig2)

# Test case 3: Numerical stability with extreme values (should not overflow)
sig3 = _sigmoid(z3_extreme)
check_sigmoid_dtypes_shape(sig3, z3_extreme)
check_sigmoid_range(sig3)
# For very large positive z, sigmoid -> 1; for very large negative z, sigmoid -> 0
assert sig3[0] > 0.99, "Sigmoid of large positive value should be close to 1."
assert sig3[1] < 0.01, "Sigmoid of large negative value should be close to 0."

# Test case 4: Sigmoid at z=0 should be exactly 0.5
sig_zero = _sigmoid(np.array([0.0]))
assert np.isclose(sig_zero[0], 0.5, rtol=1e-10), "Sigmoid(0) should be 0.5."

print("All _sigmoid tests passed!")

# =====
# Unit Test for OVRClassifier (One-vs-Rest Representation)
# =====
# OVR trains K binary classifiers, where each classifier k distinguishes
# class k from all other classes combined.
# Binary label transformation: y_i'(k) = 1 if y_i == k, else 0

X1_ovr_init = np.array([[1, 2], [3, 4], [5, 6], [7, 8], [9, 10], [11, 12]])
y1_ovr_init = np.array([0, 0, 1, 1, 2, 2])

X2_ovr_init = np.array([[10, 20], [30, 40], [50, 60], [70, 80], [90, 100]])
y2_ovr_init = np.array([1, 0, 1, 0, 1])

ovr1 = OVRClassifier()
ovr2 = OVRClassifier()

ovr1.fit(X1_ovr_init, y1_ovr_init)
pred1_ovr = ovr1.predict(X1_ovr_init)

ovr2.fit(X2_ovr_init, y2_ovr_init)
pred2_ovr = ovr2.predict(X2_ovr_init)

def check_ovr_classes_dtype_shape(ovr, y_orig):
    """Check that classes_ attribute is numpy array with correct number of classes."""
    classes = np.unique(y_orig)

```



```

    assert isinstance(ovr.classes_, np.ndarray), "classes_ is not numpy array."
    assert ovr.classes_.shape == (len(classes)), "classes_ has incorrect shape."

def check_ovr_num_classifiers(ovr, y_orig):
    """Check that OVR has exactly K binary classifiers (one per class)."""
    classes = np.unique(y_orig)
    K = len(classes)
    # Each class should have a corresponding model with key 'ova_{cls}'
    for cls in classes:
        key = f'ova_{cls}'
        assert key in ovr.loss.models, f"Missing binary classifier for class {cls}."
    assert len([k for k in ovr.loss.models.keys() if k.startswith('ova_')]) == K, \
        f"Expected {K} OVA classifiers, got different count."

def check_ovr_binary_label_transformation(y_orig, cls):
    """Verify OVR binary label transformation: y_binary = 1 if y == cls, else 0."""
    y_binary = (y_orig == cls).astype(np.int64)
    # Check binary labels are in {0, 1}
    assert set(np.unique(y_binary)).issubset([0, 1]), "Binary labels not in {0, 1}."
    # Check positive label count matches original class count
    assert np.sum(y_binary == 1) == np.sum(y_orig == cls), "Positive label count mismatch."
    # Check negative label count
    assert np.sum(y_binary == 0) == np.sum(y_orig != cls), "Negative label count mismatch."
    return y_binary

def check_ovr_predict_dtype_shape(pred, X_orig):
    """Check that predict output is numpy array with correct shape."""
    assert isinstance(pred, np.ndarray), "Prediction is not numpy array."
    assert pred.shape == (X_orig.shape[0],), "Prediction has incorrect shape."

def check_ovr_predict_vals(test_vals, true_vals, rtol=1e-2):
    """Check that predicted values match expected values."""
    assert np.all(np.isclose(test_vals, true_vals, rtol=rtol)), "OVR prediction values mismatch."

# Test OVR1: 3-class problem
check_ovr_classes_dtype_shape(ovr1, y1_ovr_init)
check_ovr_num_classifiers(ovr1, y1_ovr_init)
check_ovr_predict_dtype_shape(pred1_ovr, X1_ovr_init)

# Verify binary label transformation for each class
for cls in np.unique(y1_ovr_init):
    y_bin = check_ovr_binary_label_transformation(y1_ovr_init, cls)

true_pred1_ovr = np.array([0, 0, 1, 1, 2, 2])
check_ovr_predict_vals(pred1_ovr, true_pred1_ovr)

# Test OVR2: 2-class problem
check_ovr_classes_dtype_shape(ovr2, y2_ovr_init)
check_ovr_num_classifiers(ovr2, y2_ovr_init)
check_ovr_predict_dtype_shape(pred2_ovr, X2_ovr_init)

for cls in np.unique(y2_ovr_init):
    y_bin = check_ovr_binary_label_transformation(y2_ovr_init, cls)

true_pred2_ovr = np.array([1, 1, 1, 1, 1])
check_ovr_predict_vals(pred2_ovr, true_pred2_ovr)

# Test OVR representation property: probabilities from K classifiers do not sum to 1
# This is a key characteristic distinguishing OVR from softmax regression
def check_ovr_prob_not_sum_to_one(ovr, X):
    """Verify that OVR probabilities do not necessarily sum to 1 (unlike softmax)."""
    n_samples = X.shape[0]
    n_classes = len(ovr.classes_)
    probas = np.zeros((n_samples, n_classes))
    for idx, cls in enumerate(ovr.classes_):
        probas[:, idx] = ovr.loss.predict_proba_binary(X, f'ova_{cls}')[:, 1]
    prob_sums = np.sum(probas, axis=1)
    # OVR probabilities typically do NOT sum to exactly 1
    # (They could by coincidence, but generally won't)
    return probas, prob_sums

probas_ovr1, sums_ovr1 = check_ovr_prob_not_sum_to_one(ovr1, X1_ovr_init)
assert probas_ovr1.shape == (X1_ovr_init.shape[0], len(np.unique(y1_ovr_init))), \
    "OVR probability matrix has incorrect shape."

print("All OVRClassifier tests passed!")

# =====
# Unit Test for OVOClassifier (One-vs-One Representation)
# =====
# OVO trains K(K-1)/2 binary classifiers, one for each pair of classes.
# For each pair (k, l), only samples with y in {k, l} are used.
# Binary label transformation: y_binary = 1 if y == k, else 0 (for class l)

X1_ovo_init = np.array([[1, 2], [3, 4], [5, 6], [7, 8], [9, 10], [11, 12]])
y1_ovo_init = np.array([0, 0, 1, 1, 2, 2])

X2_ovo_init = np.array([[10, 20], [30, 40], [50, 60], [70, 80], [90, 100]])
y2_ovo_init = np.array([1, 0, 1, 0, 1])

ovo1 = OVOClassifier()
ovo2 = OVOClassifier()

ovo1.fit(X1_ovo_init, y1_ovo_init)
pred1_ovo = ovo1.predict(X1_ovo_init)

ovo2.fit(X2_ovo_init, y2_ovo_init)
pred2_ovo = ovo2.predict(X2_ovo_init)

def check_ovo_classes_dtype_shape(ovo, y_orig):
    """Check that classes attribute is numpy array with correct shape."""
    classes = np.sort(np.unique(y_orig))
    assert isinstance(ovo.classes_, np.ndarray), "classes_ is not numpy array."
    assert ovo.classes_.shape == (len(classes)), "classes_ has incorrect shape."

def check_ovo_pairs_structure(ovo, y_orig):
    """Check that OVO has correct number of class pairs: K(K-1)/2."""
    classes = np.sort(np.unique(y_orig))
    K = len(classes)
    n_pairs_expected = K * (K - 1) // 2

    assert isinstance(ovo.pairs_, list), "pairs_ is not a list."
    assert len(ovo.pairs_) == n_pairs_expected, \
        f"Expected {n_pairs_expected} pairs, got {len(ovo.pairs_)}."

    # Verify each pair is a tuple of two different classes
    for (c1, c2) in ovo.pairs_:
        assert c1 in classes and c2 in classes, f"Invalid class in pair ({c1}, {c2})."
        assert c1 < c2, f"Pair ({c1}, {c2}) should have c1 < c2."

def check_ovo_num_classifiers(ovo, y_orig):
    """Check that OVO has exactly K(K-1)/2 binary classifiers."""
    classes = np.sort(np.unique(y_orig))
    K = len(classes)
    n_pairs = K * (K - 1) // 2

```

```

# Each pair should have a corresponding model with key 'ovo_[c1]_[c2]'
for (c1, c2) in ovo.pairs:
    key = f'ovo_[c1]_[c2]'
    assert key in ovo.loss.models, f"Missing classifier for pair ({c1}, {c2})."

ovo_keys = [k for k in ovo.loss.models.keys() if k.startswith('ovo.')]
assert len(ovo_keys) == n_pairs, f"Expected {n_pairs} OVO classifiers."

def check_ovo_pair_data_extraction(X_orig, y_orig, c1, c2):
    """Verify OVO data extraction: only samples with y in {c1, c2} are selected."""
    mask = np.logical_or(y_orig == c1, y_orig == c2)
    X_pair = X_orig[mask]
    y_pair = y_orig[mask]

    # Verify only two classes are present
    unique_classes = set(np.unique(y_pair))
    assert unique_classes.issubset({c1, c2}), \
        f"Pair data should only contain classes {c1} and {c2}, got {unique_classes}."

    # Verify sample count
    expected_count = np.sum(y_orig == c1) + np.sum(y_orig == c2)
    assert len(y_pair) == expected_count, "Pair data sample count mismatch."

    return X_pair, y_pair

def check_ovo_pairwise_label_transformation(y_pair, c1, c2):
    """Verify OVO binary label transformation: y_binary = 1 if y == c1, else 0."""
    y_binary = (y_pair == c1).astype(np.int64)

    # Check binary labels are in {0, 1}
    assert set(np.unique(y_binary)).issubset({0, 1}), "Pairwise labels not in {0, 1}."

    # Check label counts
    assert np.sum(y_binary == 1) == np.sum(y_pair == c1), "Class c1 label count mismatch."
    assert np.sum(y_binary == 0) == np.sum(y_pair == c2), "Class c2 label count mismatch."

    return y_binary

def check_ovo_predict_dtype_shape(pred, X_orig):
    """Check that predict output is numpy array with correct shape."""
    assert isinstance(pred, np.ndarray), "Prediction is not numpy array."
    assert pred.shape == (X_orig.shape[0],), "Prediction has incorrect shape."

def check_ovo_predict_vals(test_vals, true_vals, rtol=1e-2):
    """Check that predicted values match expected values."""
    assert np.all(np.isclose(test_vals, true_vals, rtol=rtol)), "OVO prediction values mismatch."

# Test OV01: 3-class problem -> 3 pairs: (0,1), (0,2), (1,2)
check_ovo_classes_dtype_shape(ovo1, y1_ovo_init)
check_ovo_pairs_structure(ovo1, y1_ovo_init)
check_ovo_num_classifiers(ovo1, y1_ovo_init)
check_ovo_predict_dtype_shape(pred1_ovo, X1_ovo_init)

# Verify data extraction and label transformation for each pair
for (c1, c2) in ovo1.pairs:
    X_pair, y_pair = check_ovo_pair_data_extraction(X1_ovo_init, y1_ovo_init, c1, c2)
    y_bin = check_ovo_pairwise_label_transformation(y_pair, c1, c2)

true_pred1_ovo = np.array([0, 0, 1, 1, 2, 2])
check_ovo_predict_vals(pred1_ovo, true_pred1_ovo)

# Test OV02: 2-class problem -> 1 pair: (0,1)
check_ovo_classes_dtype_shape(ovo2, y2_ovo_init)
check_ovo_pairs_structure(ovo2, y2_ovo_init)
check_ovo_num_classifiers(ovo2, y2_ovo_init)
check_ovo_predict_dtype_shape(pred2_ovo, X2_ovo_init)

for (c1, c2) in ovo2.pairs:
    X_pair, y_pair = check_ovo_pair_data_extraction(X2_ovo_init, y2_ovo_init, c1, c2)
    y_bin = check_ovo_pairwise_label_transformation(y_pair, c1, c2)

true_pred2_ovo = np.array([1, 1, 1, 1, 1])
check_ovo_predict_vals(pred2_ovo, true_pred2_ovo)

# Test OVO voting mechanism representation
def check_ovo_voting_structure(ovo, X):
    """Verify OVO voting: each pairwise classifier votes for one of two classes."""
    n_samples = X.shape[0]
    n_classes = len(ovo.classes_)
    cls2idx = {cls: idx for idx, cls in enumerate(ovo.classes_)}
    votes = np.zeros((n_samples, n_classes), dtype=np.int64)

    for (c1, c2) in ovo.pairs:
        pred = ovo.loss.predict_binary(X, f'ovo_[c1]_[c2]')
        # pred == 1 means vote for c1, pred == 0 means vote for c2
        votes[pred == 1, cls2idx[c1]] += 1
        votes[pred == 0, cls2idx[c2]] += 1

    # Total votes per sample should equal number of pairs
    n_pairs = len(ovo.pairs)
    vote_totals = np.sum(votes, axis=1)
    assert np.all(vote_totals == n_pairs), "Total votes per sample should equal number of pairs."

    return votes

votes_ovo1 = check_ovo_voting_structure(ovo1, X1_ovo_init)
assert votes_ovo1.shape == (X1_ovo_init.shape[0], len(np.unique(y1_ovo_init))), \
    "OVO vote matrix has incorrect shape."

print("All OV0Classifier tests passed!")

# =====
# Summary
# =====
print("\n" + "-"*60)
print("All Representation Unit Tests Passed Successfully!")
print("-"*60)
print("Tested components:")
print("  1. _sigmoid: dtype, shape, values, range [0,1], numerical stability")
print("  2. OVRClassifier: K classifiers, binary label transformation, prediction")
print("  3. OV0Classifier: K(K-1)/2 pairs, data extraction, voting mechanism")
print("-"*60)

All _sigmoid tests passed!
All OVRClassifier tests passed!
All OV0Classifier tests passed!

=====
All Representation Unit Tests Passed Successfully!
=====

Tested components:
1. _sigmoid: dtype, shape, values, range [0,1], numerical stability
2. OVRClassifier: K classifiers, binary label transformation, prediction
3. OV0Classifier: K(K-1)/2 pairs, data extraction, voting mechanism
=====

```

The following unit tests uses two fuctions to check the shape and value of each function in each class

```
#Unit Test For SklearnRandomState
from pytest import approx

rng1 = SklearnRandomState(seed=42)
rng2 = SklearnRandomState(seed=100)

X1_perm = np.array([[1, 2], [3, 4], [5, 6]])
X2_perm = np.array([10, 20, 30, 40])
X1_shuffle_init = np.array([[7, 8], [9, 10], [11, 12]])
X2_shuffle_init = np.array([5, 6, 7, 8, 9])

def check_permutation_dtypes_shape(perm_result, orig_x):
    assert isinstance(perm_result, np.ndarray), "Permutation result is not numpy array."
    assert perm_result.shape == orig_x.shape, "Permutation result has incorrect shape."

def check_permutation_vals(test_vals, true_vals, rtol=1e-6):
    assert np.all(np.isclose(test_vals, true_vals, rtol=rtol)), "Permutation values mismatch."

perm1_l = rng1.permutation(X1_perm)
check_permutation_dtypes_shape(perm1_l, X1_perm)
true_perm1_l = np.array([[1, 2], [3, 4], [5, 6]])
check_permutation_vals(perm1_l, true_perm1_l)

perm2_l = rng2.permutation(X2_perm)
check_permutation_dtypes_shape(perm2_l, X2_perm)
true_perm2_l = np.array([30, 20, 40, 10])
check_permutation_vals(perm2_l, true_perm2_l)

def check_shuffle_dtypes_shape(shuffled_x, orig_x):
    assert isinstance(shuffled_x, np.ndarray), "Shuffled array is not numpy array."
    assert shuffled_x.shape == orig_x.shape, "Shuffled array has incorrect shape."

def check_shuffle_vals(shuffled_x, true_vals, rtol=1e-6):
    assert np.all(np.isclose(shuffled_x, true_vals, rtol=rtol)), "Shuffled values mismatch."

X1_shuffle = X1_shuffle_init.copy()
rng1.shuffle(X1_shuffle)
check_shuffle_dtypes_shape(X1_shuffle, X1_shuffle_init)
true_shuffle1_l = np.array([[9, 10], [11, 12], [7, 8]])
check_shuffle_vals(X1_shuffle, true_shuffle1_l)

X2_shuffle = X2_shuffle_init.copy()
rng2.shuffle(X2_shuffle)
check_shuffle_dtypes_shape(X2_shuffle, X2_shuffle_init)
true_shuffle2_l = np.array([6, 9, 8, 7, 5])
check_shuffle_vals(X2_shuffle, true_shuffle2_l)
```

```
#Unit Test for train_test_split_exact
X1_init = np.array([[1, 2], [3, 4], [5, 6], [7, 8], [9, 10], [11, 12]])
y1_init = np.array([0, 0, 1, 1, 2, 2])

X2_init = np.array([[10, 20], [30, 40], [50, 60], [70, 80], [90, 100]])
y2_init = np.array([1, 1, 0, 0, 1])

X1_train, X1_test, y1_train, y1_test = train_test_split_exactly(X1_init, y1_init, test_size=0.5, random_state=42, stratify=True)
X2_train, X2_test, y2_train, y2_test = train_test_split_exactly(X2_init, y2_init, test_size=0.2, random_state=100, stratify=False)

def check_split_dtypes_shape(X_train, X_test, y_train, y_test, X_orig, y_orig, test_size):
    n_samples = X_orig.shape[0]
    n_test = int(np.floor(n_samples * test_size))
    n_train = n_samples - n_test

    assert isinstance(X_train, np.ndarray) and X_train.shape == (n_train, X_orig.shape[1]), "Train features dtype/shape error"
    assert isinstance(X_test, np.ndarray) and X_test.shape == (n_test, X_orig.shape[1]), "Test features dtype/shape error"
    assert isinstance(y_train, np.ndarray) and y_train.shape == (n_train,), "Train labels dtype/shape error"
    assert isinstance(y_test, np.ndarray) and y_test.shape == (n_test,), "Test labels dtype/shape error"

def check_split_vals(test_values, true_values, rtol=1e-6):
    assert np.all(np.isclose(test_values, true_values, rtol=rtol))

check_split_dtypes_shape(X1_train, X1_test, y1_train, y1_test, X1_init, y1_init, 0.5)
true_X1_train = np.array([[1, 2], [7, 8], [9, 10]])
true_X1_test = np.array([[3, 4], [5, 6], [11, 12]])
true_y1_train = np.array([0, 1, 2])
true_y1_test = np.array([0, 1, 2])
check_split_vals(X1_train, true_X1_train)
check_split_vals(X1_test, true_X1_test)
check_split_vals(y1_train, true_y1_train)
check_split_vals(y1_test, true_y1_test)

check_split_dtypes_shape(X2_train, X2_test, y2_train, y2_test, X2_init, y2_init, 0.2)
true_X2_train = np.array([[30, 40], [50, 60], [70, 80], [90, 100]])
true_X2_test = np.array([[10, 20]])
true_y2_train = np.array([1, 0, 0, 1])
true_y2_test = np.array([1])
check_split_vals(X2_train, true_X2_train)
check_split_vals(X2_test, true_X2_test)
check_split_vals(y2_train, true_y2_train)
check_split_vals(y2_test, true_y2_test)
```

```
#Unit Test for KFold_split
X1_init = np.array([[1, 2], [3, 4], [5, 6], [7, 8], [9, 10], [11, 12], [13, 14]])
X2_init = np.array([[10, 20], [30, 40], [50, 60], [70, 80], [90, 100], [110, 120]])

folds1 = KFold_split(X1_init, n_splits=3, random_state=42)
folds2 = KFold_split(X2_init, n_splits=2, random_state=100)

def check_kfold_dtypes_shape(folds, X_orig, n_splits):
    n_samples = X_orig.shape[0]
    fold_sizes = np.full(n_splits, n_samples // n_splits, dtype=int)
    fold_sizes[:n_samples % n_splits] += 1

    assert isinstance(folds, list), "KFold result is not list"
    assert len(folds) == n_splits, "KFold fold count mismatch"

    for idx, (train_idx, val_idx) in enumerate(folds):
        assert isinstance(train_idx, np.ndarray), f"Train indices {idx} dtype error"
        assert isinstance(val_idx, np.ndarray), f"Val indices {idx} dtype error"
        assert val_idx.shape == (fold_sizes[idx],), f"Val indices {idx} shape error"
        assert train_idx.shape == (n_samples - fold_sizes[idx],), f"Train indices {idx} shape error"

def check_kfold_vals(test_vals, true_vals, rtol=1e-6):
    assert np.all(np.isclose(test_vals, true_vals, rtol=rtol))

check_kfold_dtypes_shape(folds1, X1_init, 3)
true_folds1 = [
    (np.array([2, 4, 3, 6]), np.array([0, 1, 5])),
    (np.array([0, 1, 5, 3, 6]), np.array([2, 4])),

```

```

        (np.array([0, 1, 5, 2, 4]), np.array([3, 6]))
    ]
    for (test_train, test_val), (true_train, true_val) in zip(folds1, true_folds1):
        check_kfold_vals(test_train, true_train)
        check_kfold_vals(test_val, true_val)

    check_kfold_dtypes_shape(folds2, X2_init, 2)
    true_folds2 = [
        (np.array([3, 5, 0]), np.array([1, 2, 4])),
        (np.array([1, 2, 4]), np.array([3, 5, 0]))
    ]
    for (test_train, test_val), (true_train, true_val) in zip(folds2, true_folds2):
        check_kfold_vals(test_train, true_train)
        check_kfold_vals(test_val, true_val)

```

```

#Unit Test for StandardScaler
X1_init = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]])
X2_init = np.array([[5, 10], [15, 20], [25, 30], [35, 40], [45, 50], [55, 60]])

scaler1 = StandardScaler()
scaler2 = StandardScaler()

scaler1.fit(X1_init)
X1_transformed = scaler1.transform(X1_init)
X1_fit_transformed = scaler1.fit_transform(X1_init)

scaler2.fit(X2_init)
X2_transformed = scaler2.transform(X2_init)
X2_fit_transformed = scaler2.fit_transform(X2_init)

def check_scaler_fit_dtypes(scaler, X_orig):
    assert isinstance(scaler.mean_, np.ndarray) and scaler.mean_.shape == (X_orig.shape[1],)
    assert isinstance(scaler.scale_, np.ndarray) and scaler.scale_.shape == (X_orig.shape[1],)

def check_scaler_transform_dtypes_shape(X_trans, X_orig):
    assert isinstance(X_trans, np.ndarray) and X_trans.shape == X_orig.shape

def check_scaler_vals(test_vals, true_vals, rtol=1e-6):
    assert np.all(np.isclose(test_vals, true_vals, rtol=rtol))

check_scaler_fit_dtypes(scaler1, X1_init)
true_mean1 = np.array([5.5, 6.5, 7.5])
true_scale1 = np.array([3.35410197, 3.35410197, 3.35410197])
check_scaler_vals(scaler1.mean_, true_mean1)
check_scaler_vals(scaler1.scale_, true_scale1)

check_scaler_transform_dtypes_shape(X1_transformed, X1_init)
true_X1_transformed = np.array([[[-1.34164079, -1.34164079, -1.34164079],
                                [-0.4472136, -0.4472136, -0.4472136],
                                [0.4472136, 0.4472136, 0.4472136],
                                [1.34164079, 1.34164079, 1.34164079]]])

check_scaler_vals(X1_transformed, true_X1_transformed)
check_scaler_vals(X1_fit_transformed, true_X1_transformed)

check_scaler_fit_dtypes(scaler2, X2_init)
true_mean2 = np.array([30., 35.])
true_scale2 = np.array([17.07825128, 17.07825128])
check_scaler_vals(scaler2.mean_, true_mean2)
check_scaler_vals(scaler2.scale_, true_scale2)

check_scaler_transform_dtypes_shape(X2_transformed, X2_init)
true_X2_transformed = np.array([[[-1.46385011, -1.46385011],
                                [-0.87831007, -0.87831007],
                                [-0.29277002, -0.29277002],
                                [ 0.29277002, 0.29277002],
                                [ 0.87831007, 0.87831007],
                                [ 1.46385011, 1.46385011]]])

check_scaler_vals(X2_transformed, true_X2_transformed)
check_scaler_vals(X2_fit_transformed, true_X2_transformed)

```

```

#Unit Test for LogisticLoss

X1_init = np.array([[1, 2], [3, 4], [5, 6], [7, 8]])
y1_init = np.array([0, 0, 1, 1])

X2_init = np.array([[10, 20], [30, 40], [50, 60], [70, 80], [90, 100]])
y2_init = np.array([1, 0, 1, 0, 1])

loss1 = LogisticLoss()
loss2 = LogisticLoss()

clf1 = loss1.train_binary(X1_init, y1_init, 'test_key1')
probal = loss1.predict_proba_binary(X1_init, 'test_key1')
pred1 = loss1.predict_binary(X1_init, 'test_key1')

clf2 = loss2.train_binary(X2_init, y2_init, 'test_key2')
proba2 = loss2.predict_proba_binary(X2_init, 'test_key2')
pred2 = loss2.predict_binary(X2_init, 'test_key2')

def check_train_binary_return(clf):
    assert hasattr(clf, 'fit') and hasattr(clf, 'predict_proba') and hasattr(clf, 'predict')

def check_models_dict(loss, key):
    assert isinstance(loss.models, dict)
    assert key in loss.models.keys()

def check_predict_proba_shape(proba, X_orig):
    assert isinstance(proba, np.ndarray)
    assert proba.shape == (X_orig.shape[0], 2)

def check_predict_shape(pred, X_orig):
    assert isinstance(pred, np.ndarray)
    assert pred.shape == (X_orig.shape[0],)

def check_loss_vals(test_vals, true_vals, rtol=1e-2):
    assert np.all(np.isclose(test_vals, true_vals, rtol=rtol))

check_train_binary_return(clf1)
check_models_dict(loss1, 'test_key1')
check_predict_proba_shape(probal, X1_init)
check_predict_shape(pred1, X1_init)
true_probal = np.array([[0.97470783, 0.02529217],
                        [0.77157294, 0.22842706],
                        [0.22842706, 0.77157294],
                        [0.02529217, 0.97470783]])

true_pred1 = np.array([0, 0, 1, 1])
check_loss_vals(probal, true_probal)
check_loss_vals(pred1, true_pred1)

check_train_binary_return(clf2)
check_models_dict(loss2, 'test_key2')
check_predict_proba_shape(proba2, X2_init)
check_predict_shape(pred2, X2_init)
true_proba2 = np.array([[0.4, 0.6],

```

```

        [0.4, 0.6],
        [0.4, 0.6],
        [0.4, 0.6],
        [0.4, 0.6]])
true_pred2 = np.array([1, 1, 1, 1, 1])
check_loss_vals(proba2, true_proba2)
check_loss_vals(pred2, true_pred2)

```

```

# Unit Test For OVRClassifier
X1_init = np.array([[1, 2], [3, 4], [5, 6], [7, 8], [9, 10], [11, 12]])
y1_init = np.array([0, 0, 1, 1, 2, 2])
X2_init = np.array([[10, 20], [30, 40], [50, 60], [70, 80], [90, 100]])
y2_init = np.array([1, 0, 1, 0, 1])

ovr1 = OVRClassifier()
ovr2 = OVRClassifier()

ovr1.fit(X1_init, y1_init)
pred1 = ovr1.predict(X1_init)

ovr2.fit(X2_init, y2_init)
pred2 = ovr2.predict(X2_init)

def check_ovr_fit_dtypes(ovr, y_orig):
    assert isinstance(ovr.classes_, np.ndarray)
    assert ovr.classes_.shape == (len(np.unique(y_orig)),)
    assert isinstance(ovr.loss.models, dict)

def check_ovr_predict_shape(pred, X_orig):
    assert isinstance(pred, np.ndarray)
    assert pred.shape == (X_orig.shape[0],)

def check_ovr_vals(test_vals, true_vals, rtol=1e-2):
    assert np.all(np.isclose(test_vals, true_vals, rtol=rtol))

check_ovr_fit_dtypes(ovr1, y1_init)
check_ovr_predict_shape(pred1, X1_init)
true_pred1 = np.array([0, 0, 1, 1, 2, 2])
check_ovr_vals(pred1, true_pred1)

check_ovr_fit_dtypes(ovr2, y2_init)
check_ovr_predict_shape(pred2, X2_init)
true_pred2 = np.array([1, 1, 1, 1, 1])
check_ovr_vals(pred2, true_pred2)

```

```

# Unit Test For OVOClassifier
X1_init = np.array([[1, 2], [3, 4], [5, 6], [7, 8], [9, 10], [11, 12]])
y1_init = np.array([0, 0, 1, 1, 2, 2])

X2_init = np.array([[10, 20], [30, 40], [50, 60], [70, 80], [90, 100]])
y2_init = np.array([1, 0, 1, 0, 1])

ovo1 = OVOClassifier()
ovo2 = OVOClassifier()

ovo1.fit(X1_init, y1_init)
pred1 = ovo1.predict(X1_init)

ovo2.fit(X2_init, y2_init)
pred2 = ovo2.predict(X2_init)

def check_ovo_fit_dtypes(ovo, y_orig):
    classes = np.sort(np.unique(y_orig))
    n_classes = len(classes)
    n_pairs = n_classes * (n_classes - 1) // 2

    assert isinstance(ovo.classes_, np.ndarray)
    assert ovo.classes_.shape == (n_classes,)
    assert isinstance(ovo.pairs_, list)
    assert len(ovo.pairs_) == n_pairs
    assert isinstance(ovo.loss.models, dict)

def check_ovo_predict_shape(pred, X_orig):
    assert isinstance(pred, np.ndarray)
    assert pred.shape == (X_orig.shape[0],)

def check_ovo_vals(test_vals, true_vals, rtol=1e-2):
    assert np.all(np.isclose(test_vals, true_vals, rtol=rtol))

check_ovo_fit_dtypes(ovo1, y1_init)
check_ovo_predict_shape(pred1, X1_init)
true_pred1 = np.array([0, 0, 1, 1, 2, 2])
check_ovo_vals(pred1, true_pred1)

check_ovo_fit_dtypes(ovo2, y2_init)
check_ovo_predict_shape(pred2, X2_init)
true_pred2 = np.array([1, 1, 1, 1, 1])
check_ovo_vals(pred2, true_pred2)

```

▼ Result

```

def load_data(file_path):
    column_names = [
        'erythema', 'scaling', 'definite_borders', 'itching', 'koebner_phenomenon',
        'polygonal_papules', 'follicular_papules', 'oral_mucosal_involvement',
        'knee_elbow_involvement', 'scalp_involvement', 'family_history',
        'melanin_incontinence', 'eosinophils_in_the_infiltrate', 'PNL_infiltrate',
        'fibrosis_of_the_papillary_dermis', 'exocytosis', 'acanthosis', 'hyperkeratosis',
        'parakeratosis', 'clubbing_of_the_rete_ridges', 'elongation_of_the_rete_ridges',
        'thinning_of_the_suprapapillary_epidermis', 'spongiform_pustule', 'munro_microabcess',
        'focal_hypergranulosis', 'disappearance_of_the_granular_layer', 'vacuolisation',
        'spongiosis', 'saw-tooth_appearance_of_retes', 'follicular_horn_plug',
        'perifollicular_parakeratosis', 'inflammatory_mononuclear_infiltrate',
        'band-like_infiltrate', 'Age (linear)', 'class'
    ]
    df = pd.read_csv(file_path, header=None, names=column_names, dtype=str)
    df['Age (linear)'] = df['Age (linear)'].replace('?', np.nan)
    df['Age (linear)'] = pd.to_numeric(df['Age (linear)'], errors='coerce')
    df['Age (linear)'] = df['Age (linear)'].fillna(df['Age (linear)'].median())
    for col in df.columns[:-1]:
        df[col] = pd.to_numeric(df[col], errors='coerce').astype(np.float64)
    df['class'] = pd.to_numeric(df['class'], errors='coerce').astype(np.int64)
    X = df.drop('class', axis=1).values
    y = df['class'].values
    return X, y

```

```

FILE_PATH = "/content/drive/Shared drives/DATA2060 Final project/dermatology.data"
X, y = load_data(FILE_PATH)

```

```

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

```

```
def calculate_metrics(y_true, y_pred, average='macro'):\n    acc = accuracy_score(y_true, y_pred)\n    pre = precision_score(y_true, y_pred, average=average, zero_division=0)\n    rec = recall_score(y_true, y_pred, average=average, zero_division=0)\n    f1 = f1_score(y_true, y_pred, average=average, zero_division=0)\n    return {'acc': acc, 'pre': pre, 'rec': rec, 'f1': f1}\n\ndef main(X, y):\n    TEST_SIZE = 0.2\n    K_FOLDS = 5\n    RANDOM_SEED = 42\n    METRIC_AVERAGE = 'macro'\n\n    X_train_val, X_test, y_train_val, y_test = train_test_split_exactly(\n        X, y, test_size=TEST_SIZE, random_state=RANDOM_SEED, stratify=True\n    )\n    scaler = StandardScaler()\n    X_train_val_norm = scaler.fit_transform(X_train_val)\n    X_test_norm = scaler.transform(X_test)\n    folds = KFold_split(X_train_val_norm, n_splits=K_FOLDS, random_state=RANDOM_SEED)\n\n    ova_metrics = {'acc': [], 'pre': [], 'rec': [], 'f1': []}\n    ovo_metrics = {'acc': [], 'pre': [], 'rec': [], 'f1': []}\n\n    print(f"=== {K_FOLDS}-Fold Validation (Train+Val) ===")\n    print(f"Metric average mode: {METRIC_AVERAGE}\\n")\n\n    for fold_idx, (train_idx, val_idx) in enumerate(folds):\n        X_train = X_train_val_norm[train_idx]\n        X_val = X_train_val_norm[val_idx]\n        y_train = y_train_val[train_idx]\n        y_val = y_train_val[val_idx]\n\n        ova_clf = OVRClassifier()\n        ova_clf.fit(X_train, y_train)\n        y_pred_ova = ova_clf.predict(X_val)\n        ova_fold_metric = calculate_metrics(y_val, y_pred_ova, METRIC_AVERAGE)\n        for key in ova_metrics.keys():\n            ova_metrics[key].append(ova_fold_metric[key])\n\n        ovo_clf = OVOCClassifier()\n        ovo_clf.fit(X_train, y_train)\n        y_pred_ovo = ovo_clf.predict(X_val)\n        ovo_fold_metric = calculate_metrics(y_val, y_pred_ovo, METRIC_AVERAGE)\n        for key in ovo_metrics.keys():\n            ovo_metrics[key].append(ovo_fold_metric[key])\n\n        print(f"Fold {fold_idx+1}/{K_FOLDS}")\n        print(f"OvA - Acc: {ova_fold_metric['acc']:.4f}, Precision: {ova_fold_metric['pre']:.4f}, Recall: {ova_fold_metric['rec']:.4f}, F1: {ova_fold_metric['f1']:.4f}")\n        print(f"OvO - Acc: {ovo_fold_metric['acc']:.4f}, Precision: {ovo_fold_metric['pre']:.4f}, Recall: {ovo_fold_metric['rec']:.4f}, F1: {ovo_fold_metric['f1']:.4f}\\n")\n\n    ova_mean = {k: np.mean(v) for k, v in ova_metrics.items()}\n    ova_std = {k: np.std(v) for k, v in ova_metrics.items()}\n    ovo_mean = {k: np.mean(v) for k, v in ovo_metrics.items()}\n    ovo_std = {k: np.std(v) for k, v in ovo_metrics.items()}\n\n    print(f"~*~*~80)\n    print(f"OvA Mean Val Metrics (±std):")\n    print(f" Acc: {ova_mean['acc']:.4f} (±{ova_std['acc']:.4f}), Precision: {ova_mean['pre']:.4f} (±{ova_std['pre']:.4f})")\n    print(f" Recall: {ova_mean['rec']:.4f} (±{ova_std['rec']:.4f}), F1: {ova_mean['f1']:.4f} (±{ova_std['f1']:.4f})")\n\n    print(f"~nOvO Mean Val Metrics (±std):")\n    print(f" Acc: {ovo_mean['acc']:.4f} (±{ovo_std['acc']:.4f}), Precision: {ovo_mean['pre']:.4f} (±{ovo_std['pre']:.4f})")\n    print(f" Recall: {ovo_mean['rec']:.4f} (±{ovo_std['rec']:.4f}), F1: {ovo_mean['f1']:.4f} (±{ovo_std['f1']:.4f})\\n")\n\n    print(f"=== Final Evaluation on Test Set ===")\n    ova_clf_final = OVRClassifier()\n    ova_clf_final.fit(X_train_val_norm, y_train_val)\n    y_pred_ova_test = ova_clf_final.predict(X_test_norm)\n    ova_test_metric = calculate_metrics(y_test, y_pred_ova_test, METRIC_AVERAGE)\n\n    ovo_clf_final = OVOCClassifier()\n    ovo_clf_final.fit(X_train_val_norm, y_train_val)\n    y_pred_ovo_test = ovo_clf_final.predict(X_test_norm)\n    ovo_test_metric = calculate_metrics(y_test, y_pred_ovo_test, METRIC_AVERAGE)\n\n    print(f"OvA Test Metrics:")\n    print(f" Acc: {ova_test_metric['acc']:.4f}, Precision: {ova_test_metric['pre']:.4f}, Recall: {ova_test_metric['rec']:.4f}, F1: {ova_test_metric['f1']:.4f}")\n\n    print(f"~nOvO Test Metrics:")\n    print(f" Acc: {ovo_test_metric['acc']:.4f}, Precision: {ovo_test_metric['pre']:.4f}, Recall: {ovo_test_metric['rec']:.4f}, F1: {ovo_test_metric['f1']:.4f}\\n")\n\n    print(f"=== Full Results Summary ===")\n    print(f"OvA - Val: Acc={ova_mean['acc']:.4f}, Pre={ova_mean['pre']:.4f}, Rec={ova_mean['rec']:.4f}, F1={ova_mean['f1']:.4f} | Test: Acc={ova_test_metric['acc']:.4f}, Pre={ova_test_metric['pre']:.4f},\n    print(f"OvO - Val: Acc={ovo_mean['acc']:.4f}, Pre={ovo_mean['pre']:.4f}, Rec={ovo_mean['rec']:.4f}, F1={ovo_mean['f1']:.4f} | Test: Acc={ovo_test_metric['acc']:.4f}, Pre={ovo_test_metric['pre']:.4f},\n    print(f"~nTest set size: {len(X_test)} samples (20% of total)")\n    print(f"~nTrain+Val set size: {len(X_train_val)} samples (80% of total)")\n\nif __name__ == "__main__":\n\n    main(X, y)
```

```
=== 5-Fold Validation (Train+Val) ===\nMetric average mode: macro\n\nFold 1/5\nOvA - Acc: 0.9661, Precision: 0.9528, Recall: 0.9528, F1: 0.9528\nOvO - Acc: 0.9661, Precision: 0.9528, Recall: 0.9528, F1: 0.9528\n\nFold 2/5\nOvA - Acc: 0.9831, Precision: 0.9722, Recall: 0.9902, F1: 0.9798\nOvO - Acc: 0.9831, Precision: 0.9722, Recall: 0.9902, F1: 0.9798\n\nFold 3/5\nOvA - Acc: 0.9661, Precision: 0.9630, Recall: 0.9630, F1: 0.9630\nOvO - Acc: 0.9831, Precision: 0.9833, Recall: 0.9815, F1: 0.9814\n\nFold 4/5\nOvA - Acc: 1.0000, Precision: 1.0000, Recall: 1.0000, F1: 1.0000\nOvO - Acc: 1.0000, Precision: 1.0000, Recall: 1.0000, F1: 1.0000\n\nFold 5/5\nOvA - Acc: 0.9828, Precision: 0.9848, Recall: 0.9872, F1: 0.9854\nOvO - Acc: 0.9828, Precision: 0.9848, Recall: 0.9872, F1: 0.9854
```

```
-----\nOvA Mean Val Metrics (±std):\nAcc: 0.9796 (±0.0127), Precision: 0.9746 (±0.0165)\nRecall: 0.9786 (±0.0178), F1: 0.9762 (±0.0167)\n\nOvO Mean Val Metrics (±std):\nAcc: 0.9830 (±0.0107), Precision: 0.9786 (±0.0157)\nRecall: 0.9823 (±0.0159), F1: 0.9799 (±0.0153)
```

```
=== Final Evaluation on Test Set ===\nOvA Test Metrics:\nAcc: 0.9589, Precision: 0.9594, Recall: 0.9537, F1: 0.9562
```

```
OvO Test Metrics:\nAcc: 0.9589, Precision: 0.9515, Recall: 0.9537, F1: 0.9519
```

```
== Full Results Summary ==
OvA - Val: Acc=0.9796, Pre=0.9746, Rec=0.9786, F1=0.9762 | Test: Acc=0.9589, Pre=0.9594, Rec=0.9537, F1=0.9562
Ov0 - Val: Acc=0.9830, Pre=0.9786, Rec=0.9823, F1=0.9799 | Test: Acc=0.9589, Pre=0.9515, Rec=0.9537, F1=0.9519

Test set size: 73 samples (20% of total)
Train+Val set size: 293 samples (80% of total)
```

```
import warnings
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.multiclass import OneVsRestClassifier, OneVsOneClassifier
from sklearn.exceptions import ConvergenceWarning

warnings.filterwarnings("ignore", category=ConvergenceWarning)
warnings.filterwarnings("ignore", message="Line Search failed")

def calculate_multiclass_metrics(y_true, y_pred, average='macro'):
    acc = accuracy_score(y_true, y_pred)
    pre = precision_score(y_true, y_pred, average=average, zero_division=0)
    rec = recall_score(y_true, y_pred, average=average, zero_division=0)
    f1 = f1_score(y_true, y_pred, average=average, zero_division=0)
    return {'acc': acc, 'pre': pre, 'rec': rec, 'f1': f1}

K_FOLDS = 5
TEST_SIZE = 0.2
RANDOM_SEED = 42
METRIC_AVERAGE = 'macro'

X_train_val, X_test, y_train_val, y_test = train_test_split_exactly(
    X, y, test_size=TEST_SIZE, random_state=RANDOM_SEED, stratify=True
)

scaler = StandardScaler()
X_train_val_norm = scaler.fit_transform(X_train_val)
X_test_norm = scaler.transform(X_test)

base_clf = LogisticRegression(C=1.0, solver='newton-cg', max_iter=1000, tol=1e-15)
ova_clf = OneVsRestClassifier(base_clf)
ovo_clf = OneVsOneClassifier(base_clf)

ova_fold_metrics = {'acc': [], 'pre': [], 'rec': [], 'f1': []}
ovo_fold_metrics = {'acc': [], 'pre': [], 'rec': [], 'f1': []}

folds = KFold_split(X_train_val_norm, n_splits=K_FOLDS, random_state=RANDOM_SEED)
print(f"=== {K_FOLDS}-Fold Validation (Train+Val) ===")
print(f"Metric average mode: {METRIC_AVERAGE}\n")

for fold_idx, (train_idx, val_idx) in enumerate(folds):
    X_train, X_val = X_train_val_norm[train_idx], X_train_val_norm[val_idx]
    y_train, y_val = y_train_val[train_idx], y_train_val[val_idx]

    ova_clf.fit(X_train, y_train)
    y_pred_ova = ova_clf.predict(X_val)
    ova_metric = calculate_multiclass_metrics(y_val, y_pred_ova, METRIC_AVERAGE)
    for key in ova_fold_metrics.keys():
        ova_fold_metrics[key].append(ova_metric[key])

    ovo_clf.fit(X_train, y_train)
    y_pred_ovo = ovo_clf.predict(X_val)
    ovo_metric = calculate_multiclass_metrics(y_val, y_pred_ovo, METRIC_AVERAGE)
    for key in ovo_fold_metrics.keys():
        ovo_fold_metrics[key].append(ovo_metric[key])

    print(f"Fold {fold_idx+1}/{K_FOLDS}")
    print(f"OvA - Acc: {ova_metric['acc']:.4f}, Precision: {ova_metric['pre']:.4f}, Recall: {ova_metric['rec']:.4f}, F1: {ova_metric['f1']:.4f}")
    print(f"Ov0 - Acc: {ovo_metric['acc']:.4f}, Precision: {ovo_metric['pre']:.4f}, Recall: {ovo_metric['rec']:.4f}, F1: {ovo_metric['f1']:.4f}\n")

ova_mean = {k: np.mean(v) for k, v in ova_fold_metrics.items()}
ova_std = {k: np.std(v) for k, v in ova_fold_metrics.items()}
ovo_mean = {k: np.mean(v) for k, v in ovo_fold_metrics.items()}
ovo_std = {k: np.std(v) for k, v in ovo_fold_metrics.items()}

print(f"~*80")
print(f"OvA Mean Val Metrics (±std):")
print(f"Acc: {ova_mean['acc']:.4f} (± {ova_std['acc']:.4f}), Precision: {ova_mean['pre']:.4f} (± {ova_std['pre']:.4f})")
print(f"Recall: {ova_mean['rec']:.4f} (± {ova_std['rec']:.4f}), F1: {ova_mean['f1']:.4f} (± {ova_std['f1']:.4f})")

print(f"\nOv0 Mean Val Metrics (±std):")
print(f"Acc: {ovo_mean['acc']:.4f} (± {ovo_std['acc']:.4f}), Precision: {ovo_mean['pre']:.4f} (± {ovo_std['pre']:.4f})")
print(f"Recall: {ovo_mean['rec']:.4f} (± {ovo_std['rec']:.4f}), F1: {ovo_mean['f1']:.4f} (± {ovo_std['f1']:.4f})")

print("\n=== Final Evaluation on Test Set ===")
ova_clf.fit(X_train_val_norm, y_train_val)
ovo_clf.fit(X_train_val_norm, y_train_val)

y_pred_ova_test = ova_clf.predict(X_test_norm)
ova_test_metric = calculate_multiclass_metrics(y_test, y_pred_ova_test, METRIC_AVERAGE)

y_pred_ovo_test = ovo_clf.predict(X_test_norm)
ovo_test_metric = calculate_multiclass_metrics(y_test, y_pred_ovo_test, METRIC_AVERAGE)

print(f"OvA Test Metrics:")
print(f"Acc: {ova_test_metric['acc']:.4f}, Precision: {ova_test_metric['pre']:.4f}, Recall: {ova_test_metric['rec']:.4f}, F1: {ova_test_metric['f1']:.4f}")

print(f"\nOv0 Test Metrics:")
print(f"Acc: {ovo_test_metric['acc']:.4f}, Precision: {ovo_test_metric['pre']:.4f}, Recall: {ovo_test_metric['rec']:.4f}, F1: {ovo_test_metric['f1']:.4f}")

print("\n=== Full Results Summary ===")
print(f"OvA - Val: Acc={ova_mean['acc']:.4f}, Pre={ova_mean['pre']:.4f}, Rec={ova_mean['rec']:.4f}, F1={ova_mean['f1']:.4f} | Test: Acc={ova_test_metric['acc']:.4f}, Pre={ova_test_metric['pre']:.4f}, Rec={ovo_test_metric['rec']:.4f}, F1={ova_test_metric['f1']:.4f}")
print(f"Ov0 - Val: Acc={ovo_mean['acc']:.4f}, Pre={ovo_mean['pre']:.4f}, Rec={ovo_mean['rec']:.4f}, F1={ovo_mean['f1']:.4f} | Test: Acc={ovo_test_metric['acc']:.4f}, Pre={ovo_test_metric['pre']:.4f}, Rec={ovo_test_metric['rec']:.4f}, F1={ovo_test_metric['f1']:.4f}")
print(f"Test set size: {len(X_test)} samples (20% of total)")
print(f"Train+Val set size: {len(X_train_val)} samples (80% of total)")
```

```
=== 5-Fold Validation (Train+Val) ===
Metric average mode: macro

Fold 1/5
OvA - Acc: 0.9661, Precision: 0.9528, Recall: 0.9528, F1: 0.9528
Ov0 - Acc: 0.9661, Precision: 0.9528, Recall: 0.9528, F1: 0.9528

Fold 2/5
OvA - Acc: 0.9831, Precision: 0.9722, Recall: 0.9902, F1: 0.9798
Ov0 - Acc: 0.9831, Precision: 0.9722, Recall: 0.9902, F1: 0.9798

/usr/local/lib/python3.12/dist-packages/sklearn/utils/optimize.py:100: LineSearchWarning: The line search algorithm did not converge
ret = line_search_wolfe2(
Fold 3/5
OvA - Acc: 0.9661, Precision: 0.9630, Recall: 0.9630, F1: 0.9630
Ov0 - Acc: 0.9831, Precision: 0.9833, Recall: 0.9815, F1: 0.9814

Fold 4/5
OvA - Acc: 1.0000, Precision: 1.0000, Recall: 1.0000, F1: 1.0000
Ov0 - Acc: 1.0000, Precision: 1.0000, Recall: 1.0000, F1: 1.0000

Fold 5/5
OvA - Acc: 0.9828, Precision: 0.9848, Recall: 0.9872, F1: 0.9854
Ov0 - Acc: 0.9828, Precision: 0.9848, Recall: 0.9872, F1: 0.9854
```

```
OvA Mean Val Metrics (±std):
  Acc: 0.9796 (±0.0127), Precision: 0.9746 (±0.0165)
  Recall: 0.9786 (±0.0178), F1: 0.9762 (±0.0167)

OvO Mean Val Metrics (±std):
  Acc: 0.9830 (±0.0107), Precision: 0.9786 (±0.0157)
  Recall: 0.9823 (±0.0159), F1: 0.9799 (±0.0153)

=== Final Evaluation on Test Set ===
OvA Test Metrics:
  Acc: 0.9589, Precision: 0.9594, Recall: 0.9537, F1: 0.9562

OvO Test Metrics:
  Acc: 0.9589, Precision: 0.9515, Recall: 0.9537, F1: 0.9519

=== Full Results Summary ===
OvA - Val: Acc=0.9796, Pre=0.9746, Rec=0.9786, F1=0.9762 | Test: Acc=0.9589, Pre=0.9594, Rec=0.9537, F1=0.9562
OvO - Val: Acc=0.9830, Pre=0.9786, Rec=0.9823, F1=0.9799 | Test: Acc=0.9589, Pre=0.9515, Rec=0.9537, F1=0.9519

Test set size: 73 samples (20% of total)
Train+Val set size: 293 samples (80% of total)
```

By comparing the above two outputs, we can find that the result is exactly the same, with Acc=0.9589, Pre=0.9594, Rec=0.9537, F1=0.9562 in OvA method and 0.9589, Pre=0.9515, Rec=0.9537, F1=0.9519 in OvO method.

Based on the result, the diff between two methods are not very obvious. Probaboly because the small and simple dataset can not reflect the difference.

Citation

Dermatology Dataset <https://archive.ics.uci.edu/dataset/33/dermatology>